ANSIBLE_TASK_02

1.Install httpd using ansible playbook, use handlers,notifiers.

Lab architecture

Ansible Control Node

Runs playbooks • inventory • SSH keys

Node-1

HTTPD / source file

Node-2

Tomcat / destination

Step 1: Prepare your Ansible environment

1.1 Install Ansible on the control node

For Amazon Linux 2023, check Python and install Ansible using pip in a virtual environment if the OS repository does not provide the desired version.

python3 --version

python3 -m venv ~/ansible-venv

source ~/ansible-venv/bin/activate

pip install ansible

ansible --version

On Ubuntu, you can also install Ansible from the configured package repositories:

sudo apt update

sudo apt install ansible -y

1.2 Create your project directory

mkdir -p ~/ansible-labs

cd ~/ansible-labs

mkdir -p files

1.3 Configure your inventory

Create a file named inventory.ini:

[webservers]

node1 ansible_host=10.0.1.10

[appservers]

node2 ansible_host=10.0.1.11

[all:vars]

ansible_user=ec2-user

ansible_ssh_private_key_file=~/.ssh/ansible-lab.pem

Replace the example IP addresses with your managed nodes' reachable IP addresses. For Ubuntu EC2 instances, use ansible_user=ubuntu.

Secure the private key:

chmod 400 ~/.ssh/ansible-lab.pem

Make sure the control node can reach both workers over SSH, and that their security groups allow SSH (TCP port 22) from the control node.

1.4 Test connectivity

ansible all -i inventory.ini -m ping

Expected result:

node1 | SUCCESS => {

"ping": "pong"

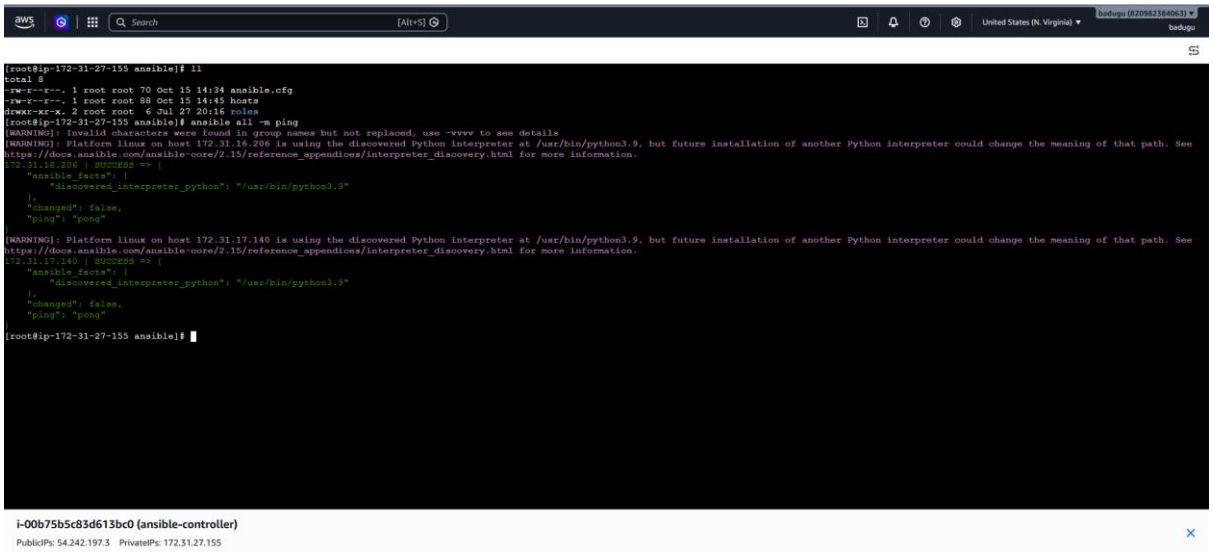
}

node2 | SUCCESS => {

"ping": "pong"

}

Ansible's ping module checks Python/Ansible connectivity; it is not an ICMP network ping.



1. Install HTTPD using an Ansible playbook with handlers and notify

Step 1.1: Create the playbook

Create install_httpd.yml

```
- name: Install and configure Apache HTTPD
  hosts: all
  become: true

  tasks:
    - name: Install HTTPD
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Create custom index page
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          <html>
          <body>
            <h1>Apache installed using Ansible</h1>
            <p>Managed by Ansible playbook.</p>
          </body>
          </html>
        owner: root
        group: root
        mode: "0644"
      notify: Restart HTTPD

    - name: Enable and start HTTPD
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true

  handlers:
    - name: Restart HTTPD
      ansible.builtin.service:
        name: httpd
        state: restarted
                                                                    0
-- INSERT --                                                        0
,1          (paste) --                                            3
7,25           All
```

i-00b75b5c83d613bc0 (ansible-controller)

PublicIPs: 54.242.197.3 PrivateIPs: 172.31.27.155

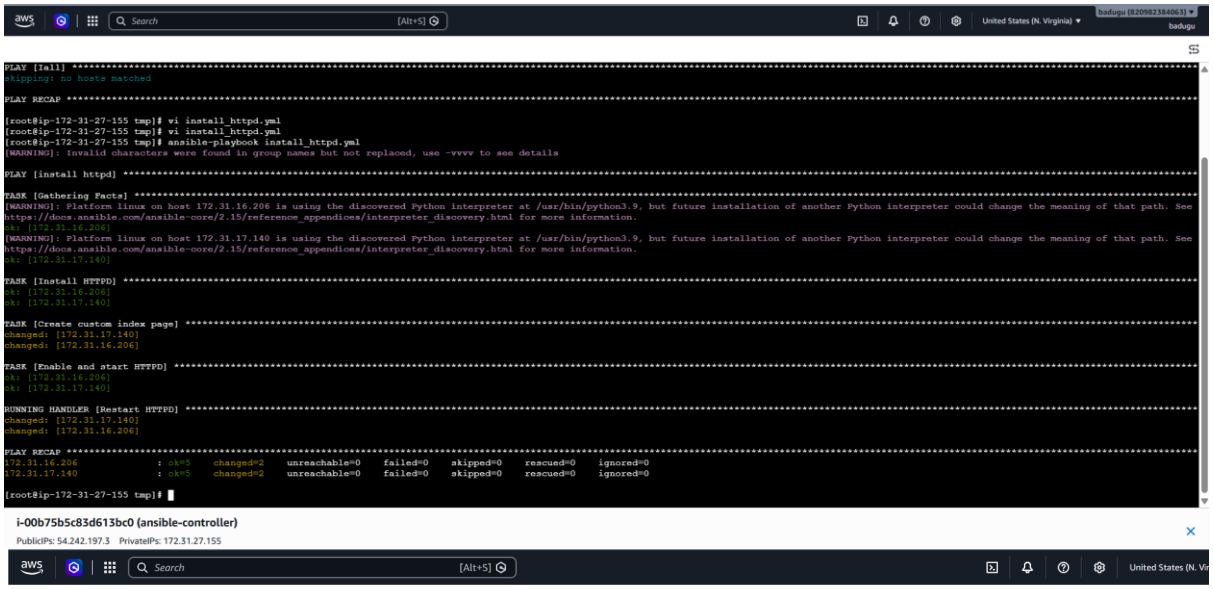
Step 1.2: Understand handlers and notify

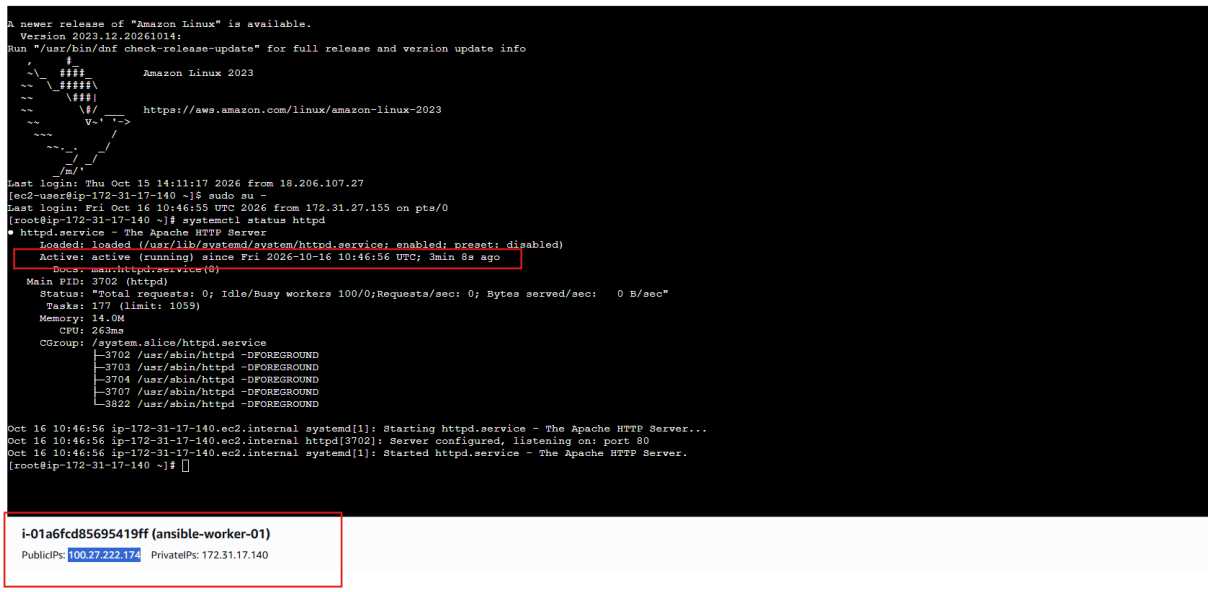
- Task: Performs an operation, such as copying a configuration file.
- notify: Signals a handler when the task reports a change.
- Handler: Runs only when notified and normally runs at the end of the play.
- Idempotency: On a second run, if the file content has not changed, the handler will not run again.

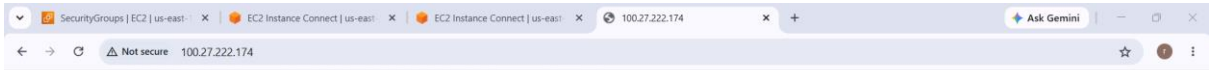
In this example, notify: Restart HTTPD calls the handler named Restart HTTPD when the index file changes. The handler is not called a notifier in Ansible; notify is the directive.

Step 1.3: Run the playbook

ansible-playbook install_httpd.yml







Apache installed using Ansible

Managed by Ansible playbook.

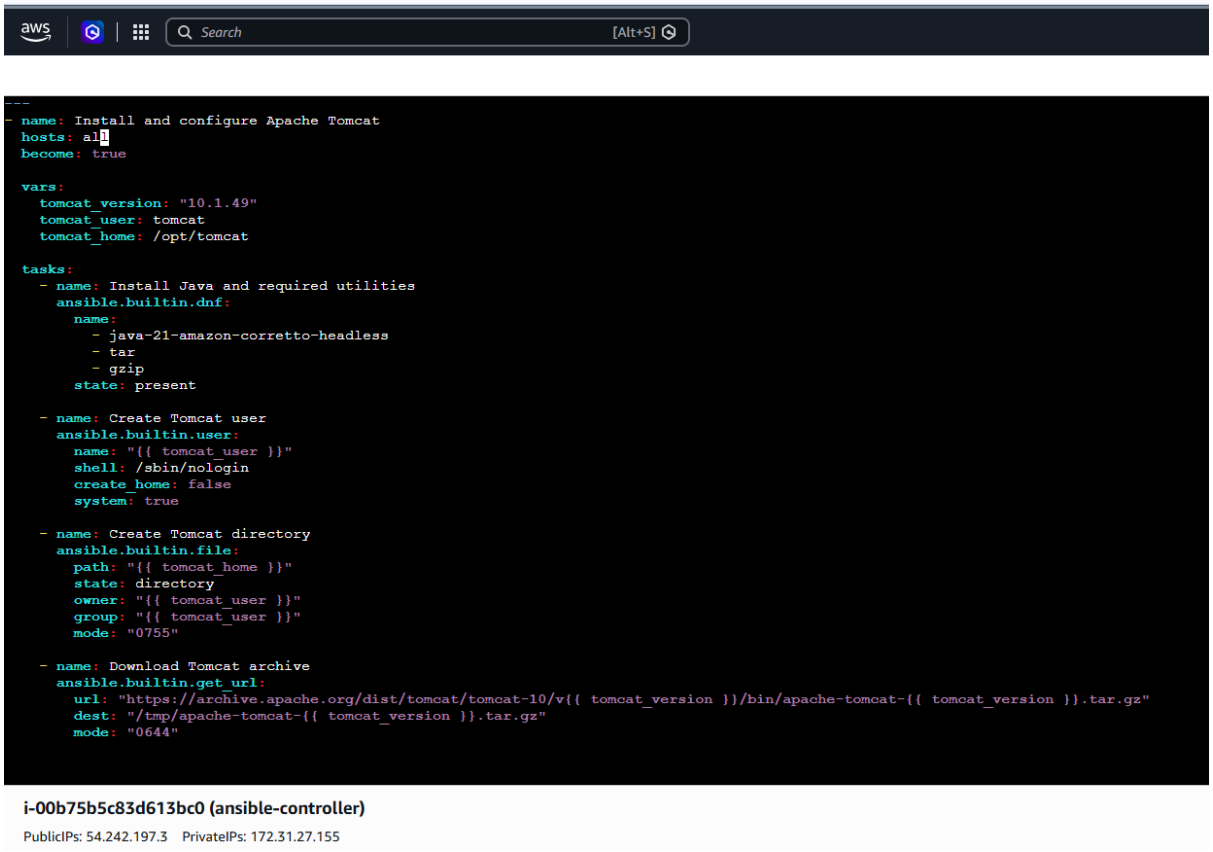
2.Write a ansible playbook to install apache tomcat.

Install Apache Tomcat using an Ansible playbook

This example installs Java, downloads Tomcat, extracts it, and configures Tomcat as a systemd service on Node-2.

Target: Amazon Linux 2023 / RHEL-style Node-2

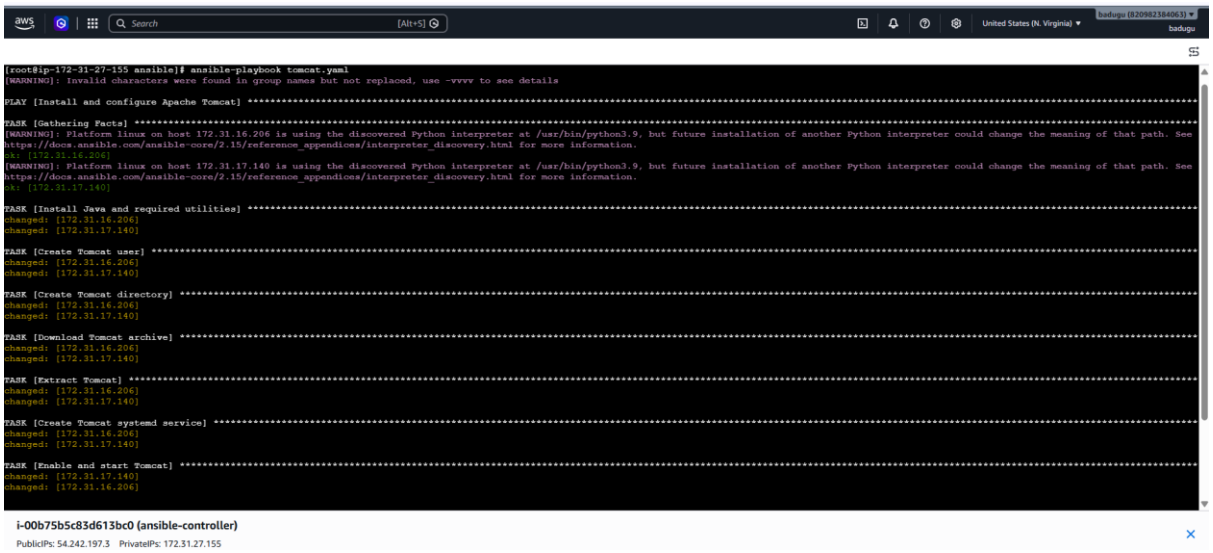
Step 2.1: Create tomcat.yml



Important notes before running:

- Verify that the selected Tomcat release is available in the Apache archive and is appropriate for your application.
- Confirm the installed Java path on your OS. If /usr/lib/jvm/java does not exist, replace JAVA_HOME with the actual Java installation directory.
- This example uses Tomcat 10.1; applications written for older Tomcat versions may require migration.

Step 2.2: Run it



Step 2.3: Verify Tomcat

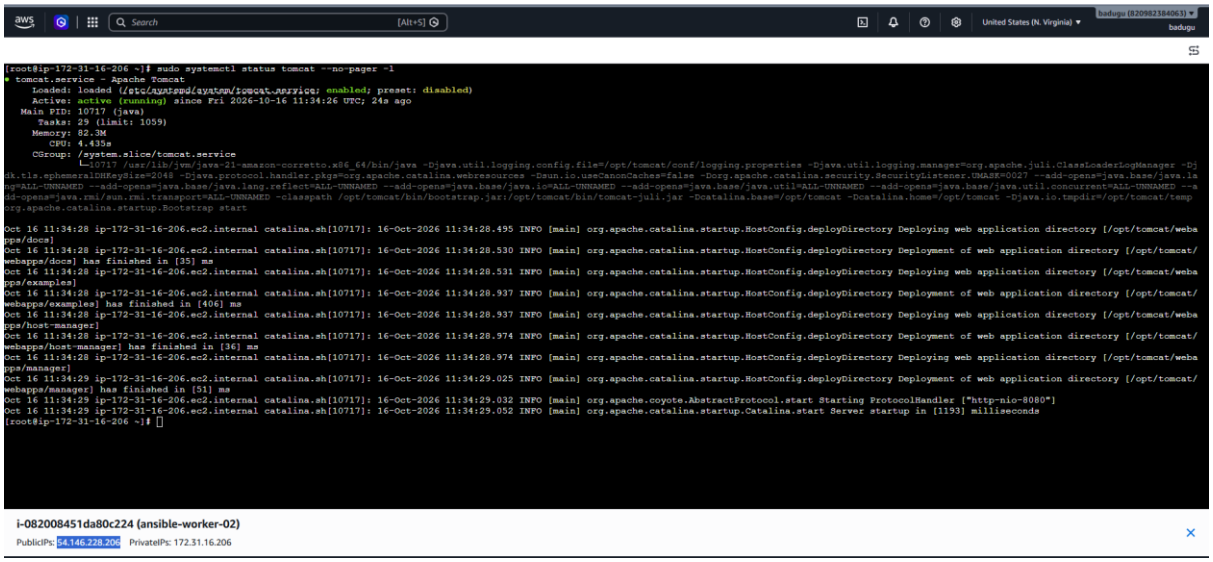
ansible appservers -i inventory.ini -b \

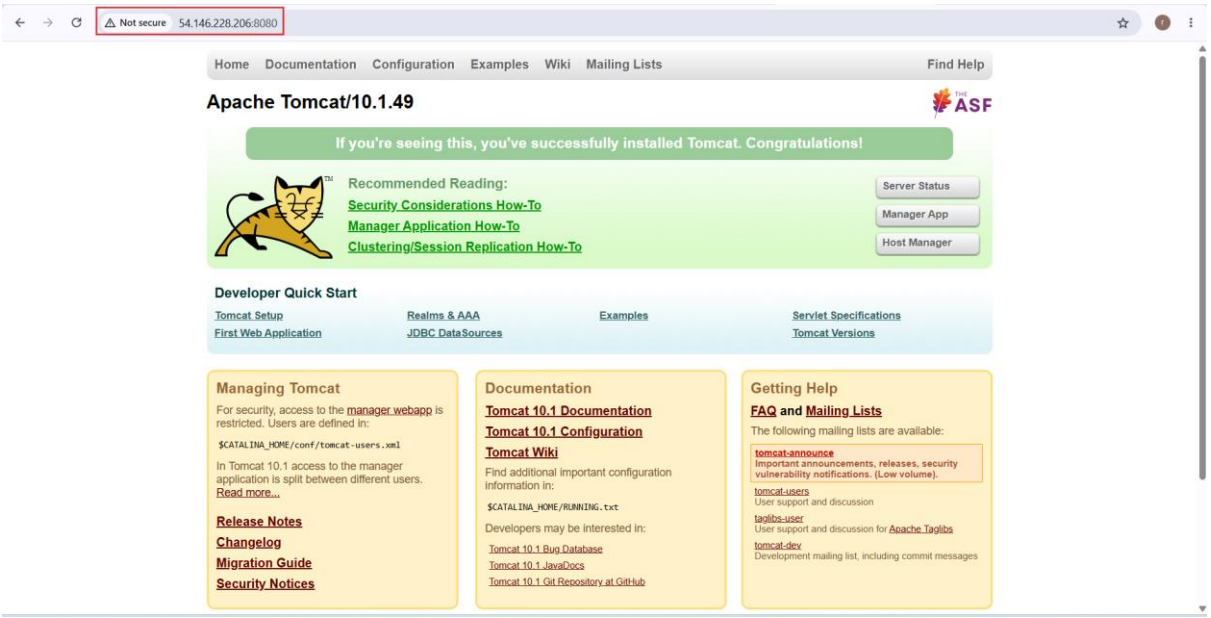
-m command -a "systemctl status tomcat --no-pager"

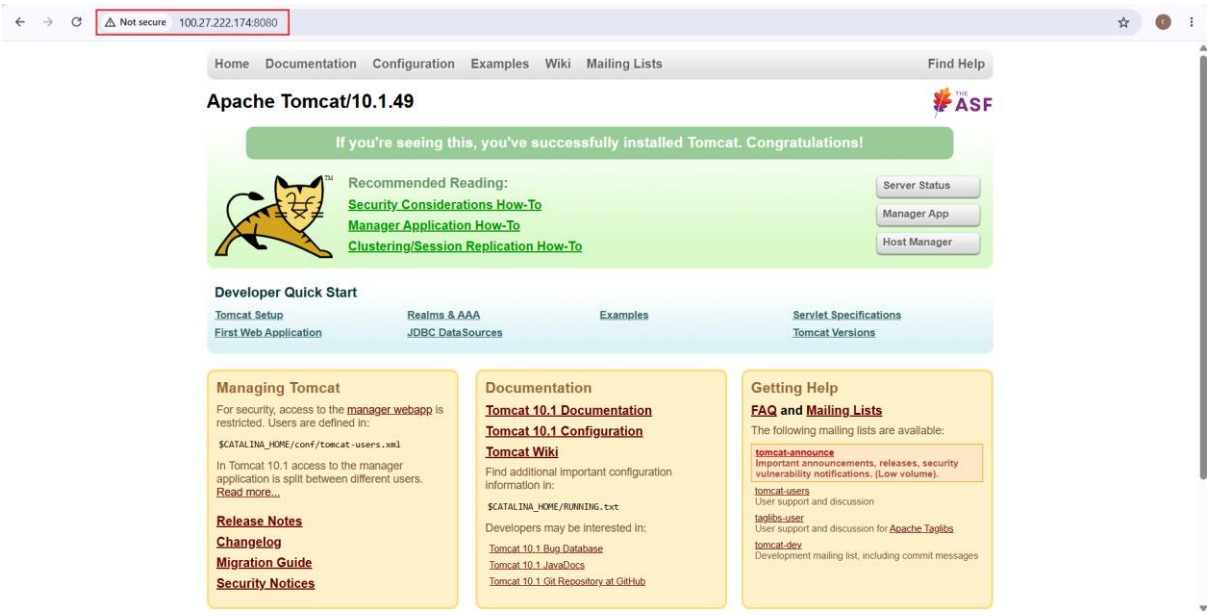
Check the local HTTP endpoint from Node-2:

curl -I http://localhost:8080

Open http://NODE2_PUBLIC_IP:8080 in your browser after allowing inbound TCP port 8080 in the security group.







3.Write a ansible playbook to provision one ec2 on aws.

Provision an EC2 instance on AWS using an Ansible playbook

For this lab, use the AWS collection amazon.aws. The playbook below launches one EC2 instance in an existing subnet and security group.

Step 3.1: Install the required collection and Python SDK

On the Ansible control node:

pip install boto3 botocore

ansible-galaxy collection install amazon.aws

If using a virtual environment, activate it before installing the packages.

If pip is missing, install it

First, try:

dnf install -y python3-pip

If dnf is unavailable, try:

yum install -y python3-pip

Then verify:

python3 -m pip –version

Install boto3 and botocore

python3 -m pip install boto3 botocore

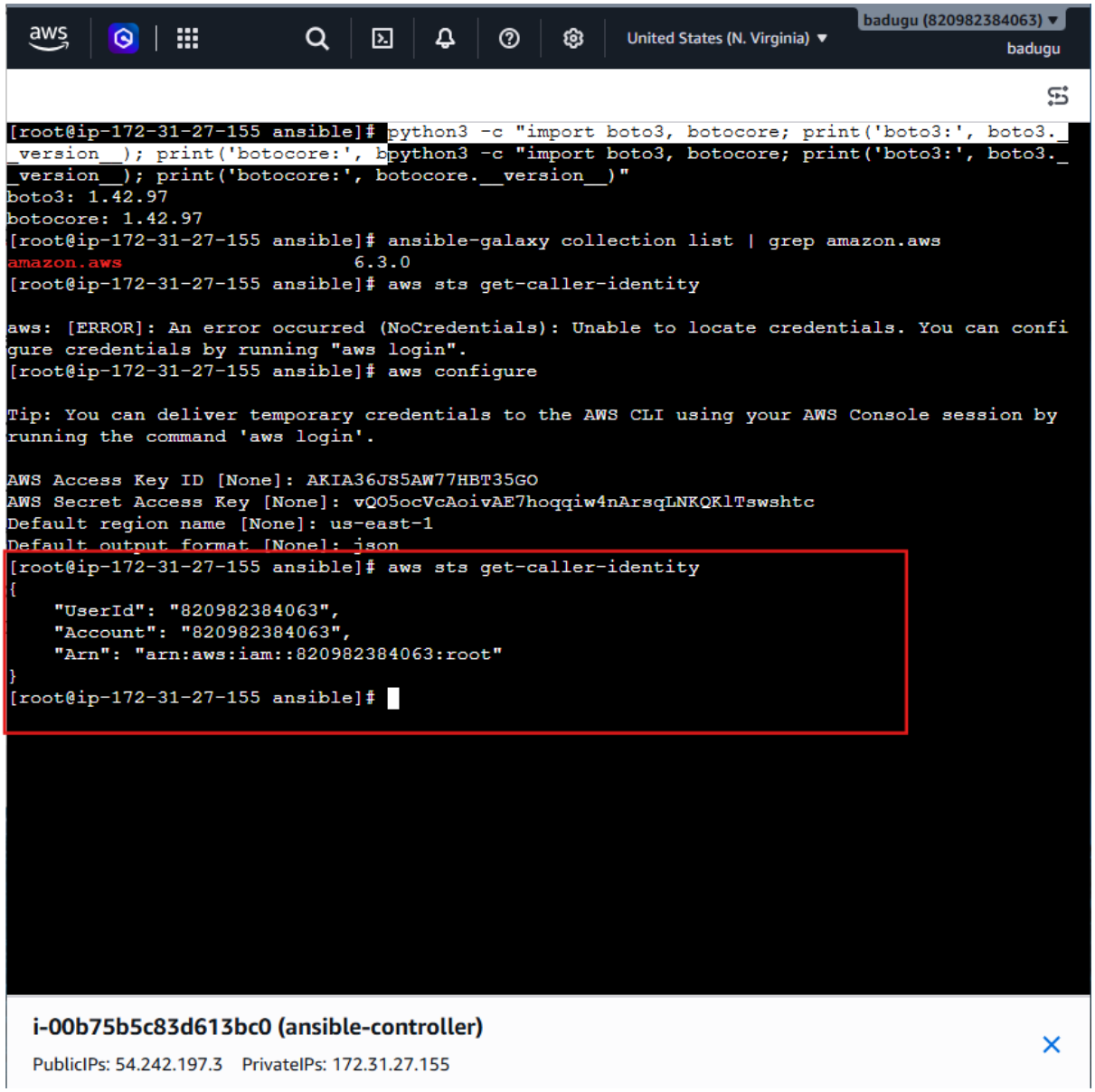
Step 3.2: Configure AWS credentials

Use an IAM role attached to the control node where possible. If running from your workstation, configure an AWS CLI profile:

aws configure --profile ansible-lab

aws sts get-caller-identity --profile ansible-lab

The identity needs permission to launch and describe EC2 instances and to use the specified subnet, security group, AMI, and key pair. Do not put AWS access keys directly in the playbook.

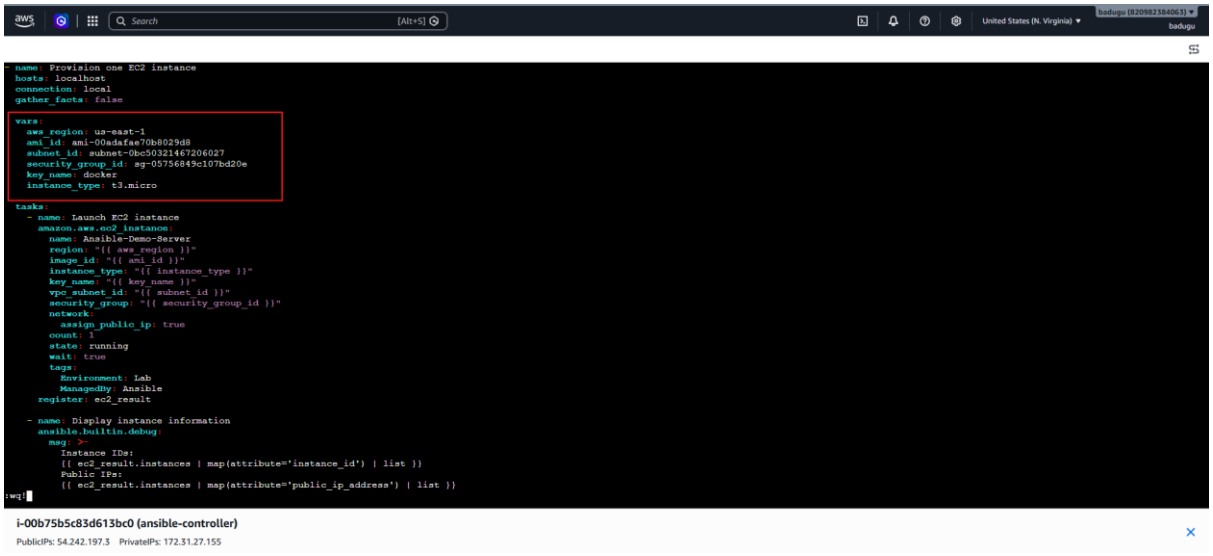


Step 3.3: Gather the AWS details

You need:

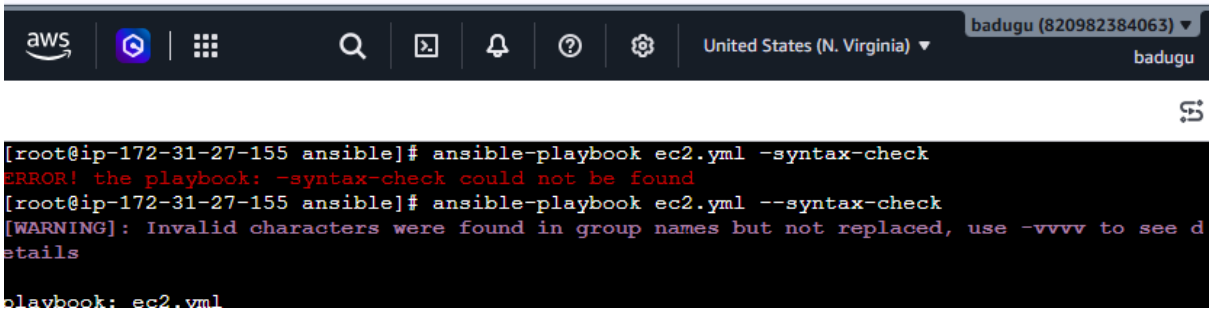
- AWS region, for example us-east-1
- A valid AMI ID for that region
- Existing subnet ID
- Existing security group ID
- Existing EC2 key pair name
- Instance type, for example t3.micro if eligible for your account

Get these values from the AWS EC2 console. The subnet must have appropriate routing if you want the instance to be publicly accessible.

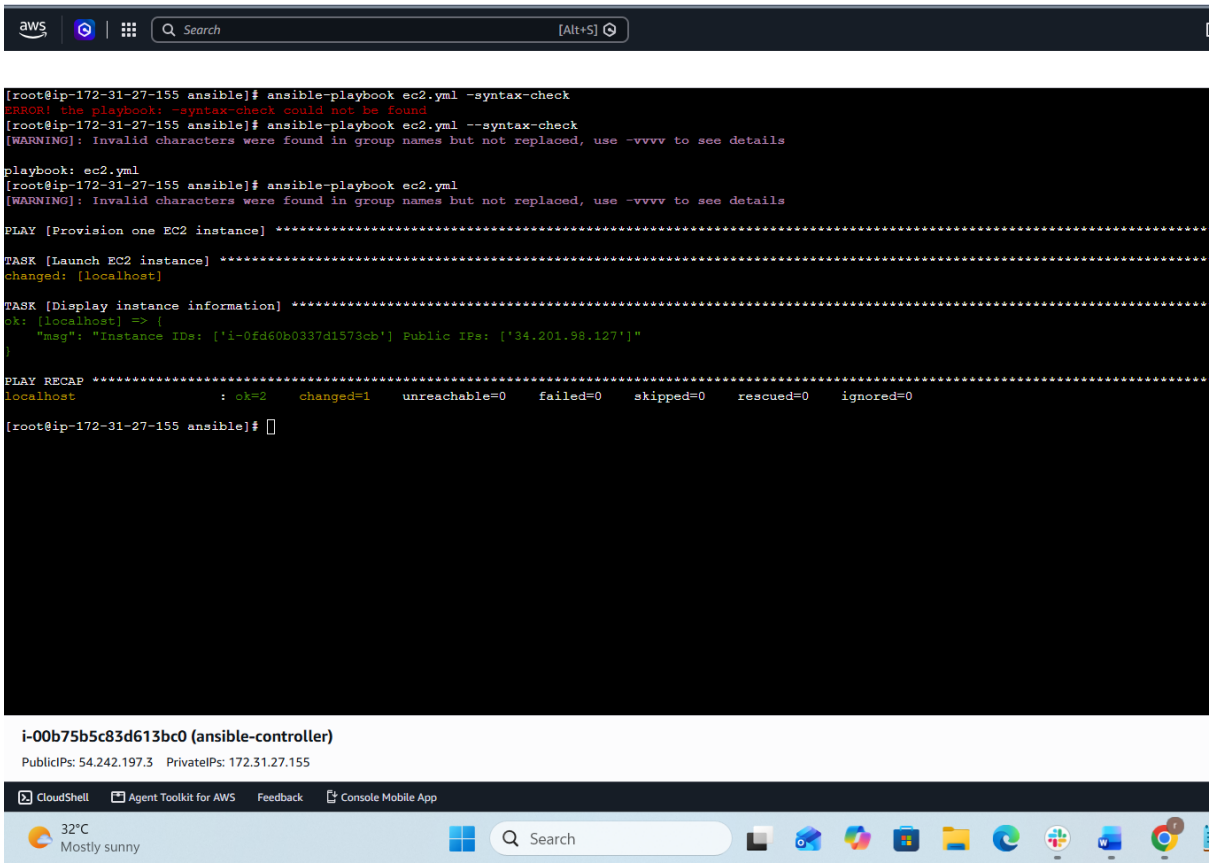


Step 3.5: Run the playbook

syntax check: # ansible-playbook ec2.yml --syntax-check



Run the playbook: # ansible-playbook create_ec2.yml

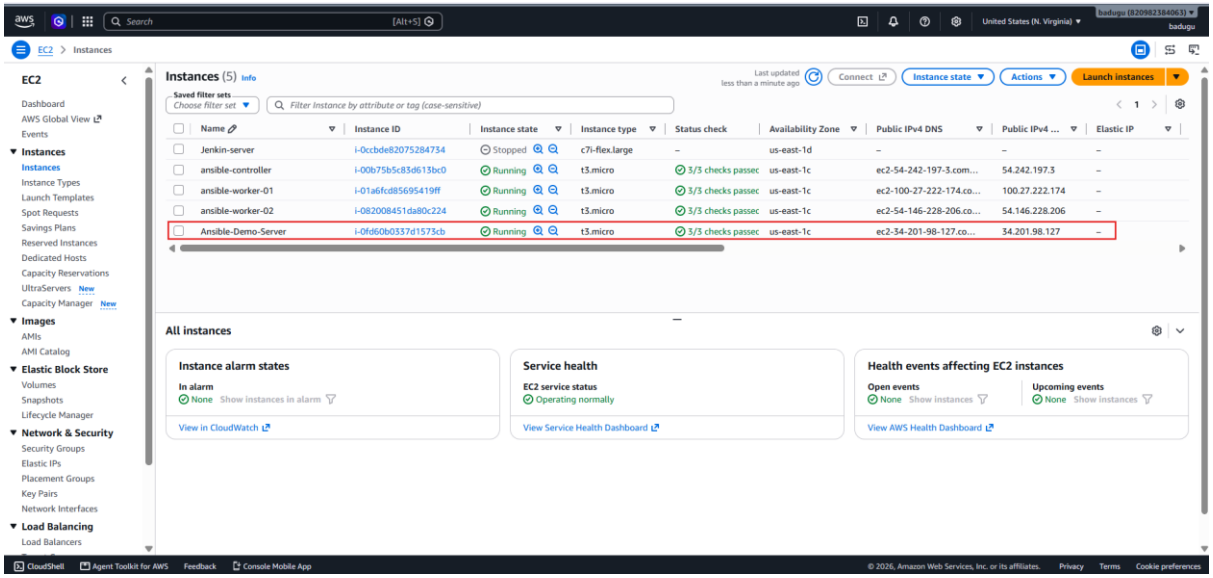


Expected output will include the instance ID and, if assigned, the public IP address.

Verify it in AWS:

1. Open the AWS Management Console.
2. Navigate to EC2 → Instances.
3. Find Ansible-Demo-Server.
4. Confirm its state is Running.

Cost and cleanup: A running EC2 instance can incur charges. When finished, terminate only the lab instance you created. If you want to make repeated runs safe, add a unique lab tag and explicitly manage the instance lifecycle rather than creating a new instance on every run.



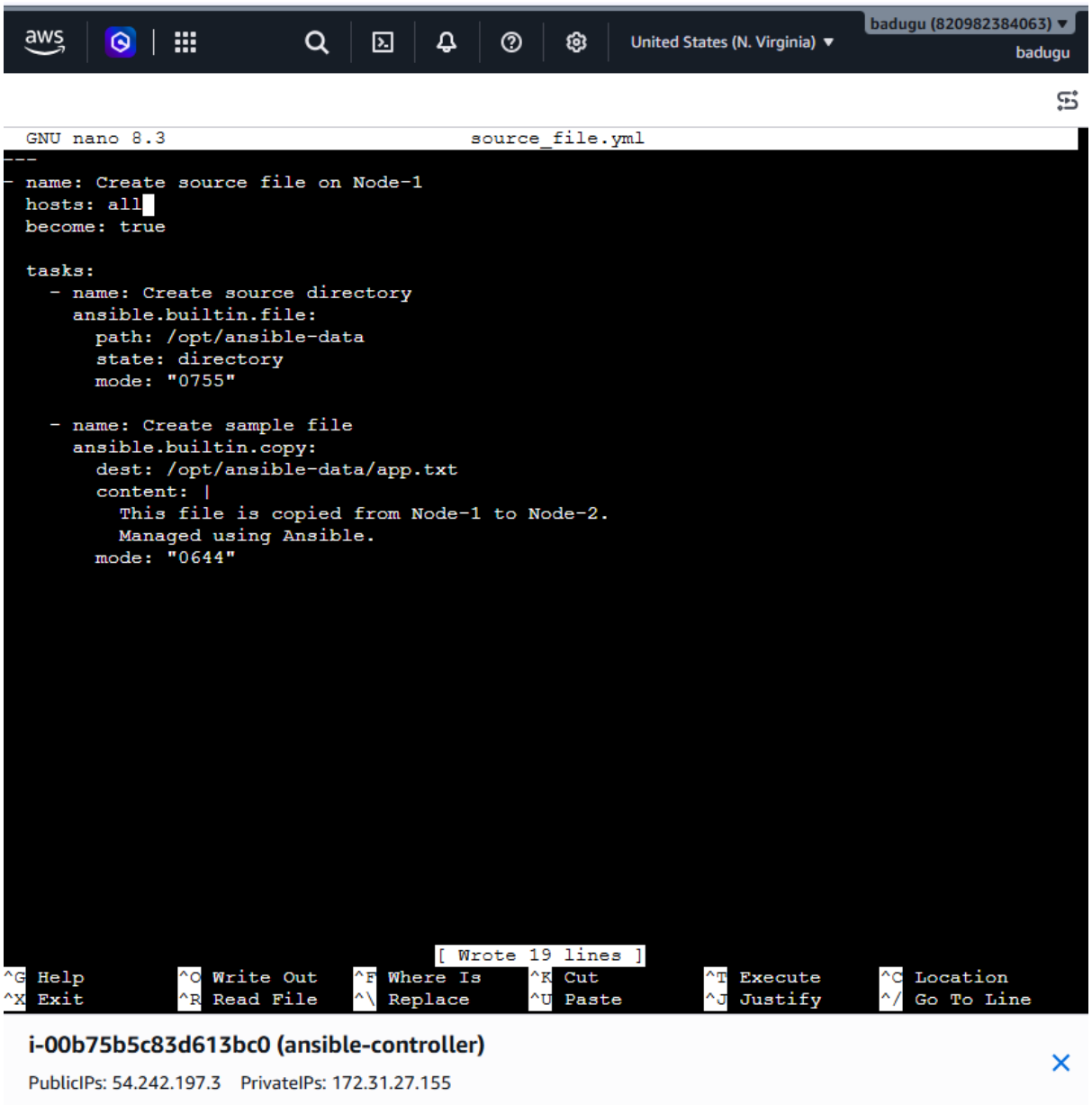
4.Write a ansible playbook to copy one file from node-1 to node-2.

Copy a file from Node-1 to Node-2 using Ansible

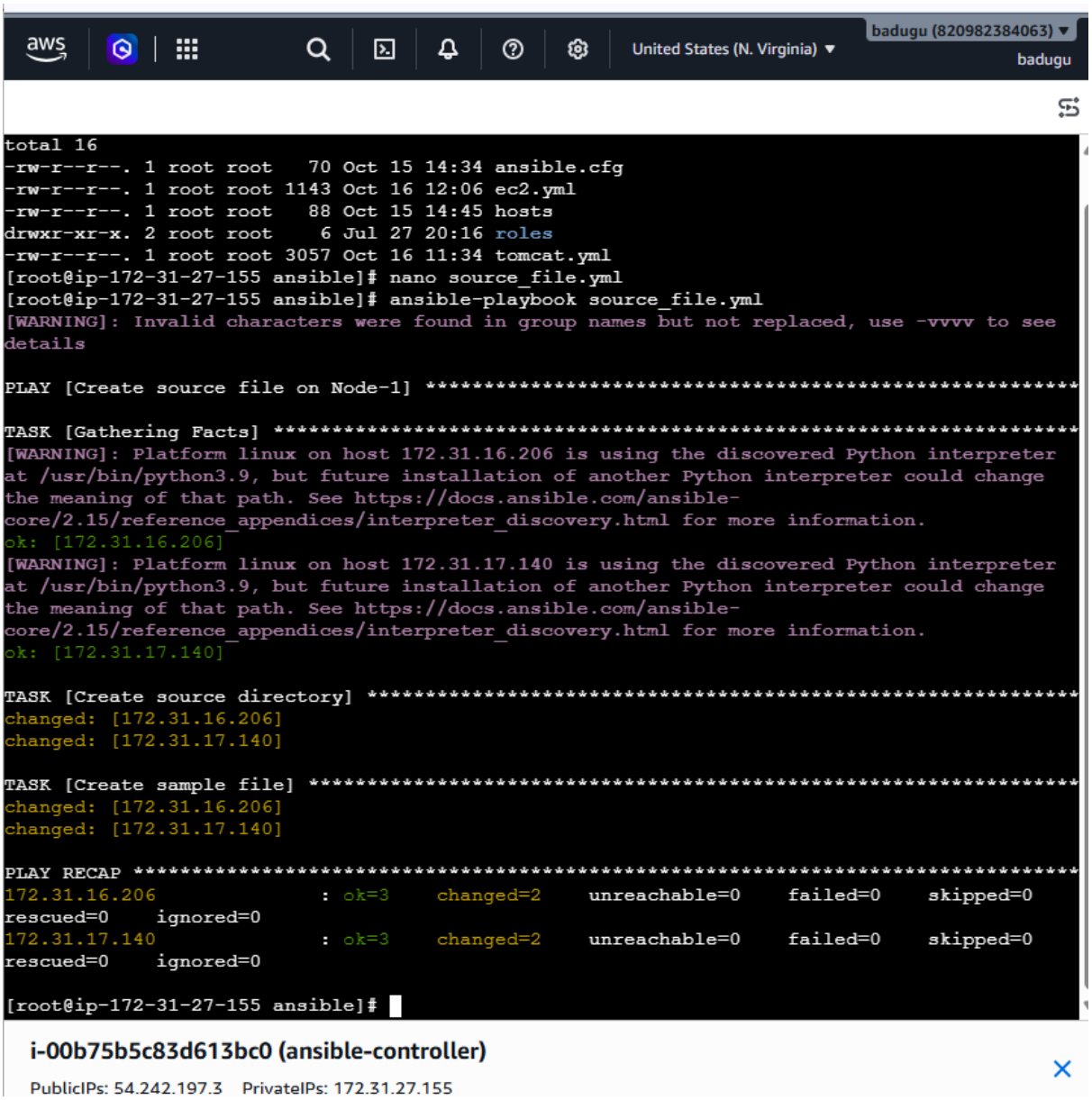
There are two common ways to copy a file between managed nodes. The easiest is to use fetch to retrieve the file from Node-1, followed by copy to send it to Node-2. This works even when Node-1 cannot SSH directly to Node-2.

Step 4.1: Create a file on Node-1

Create source_file.yml on the control node:

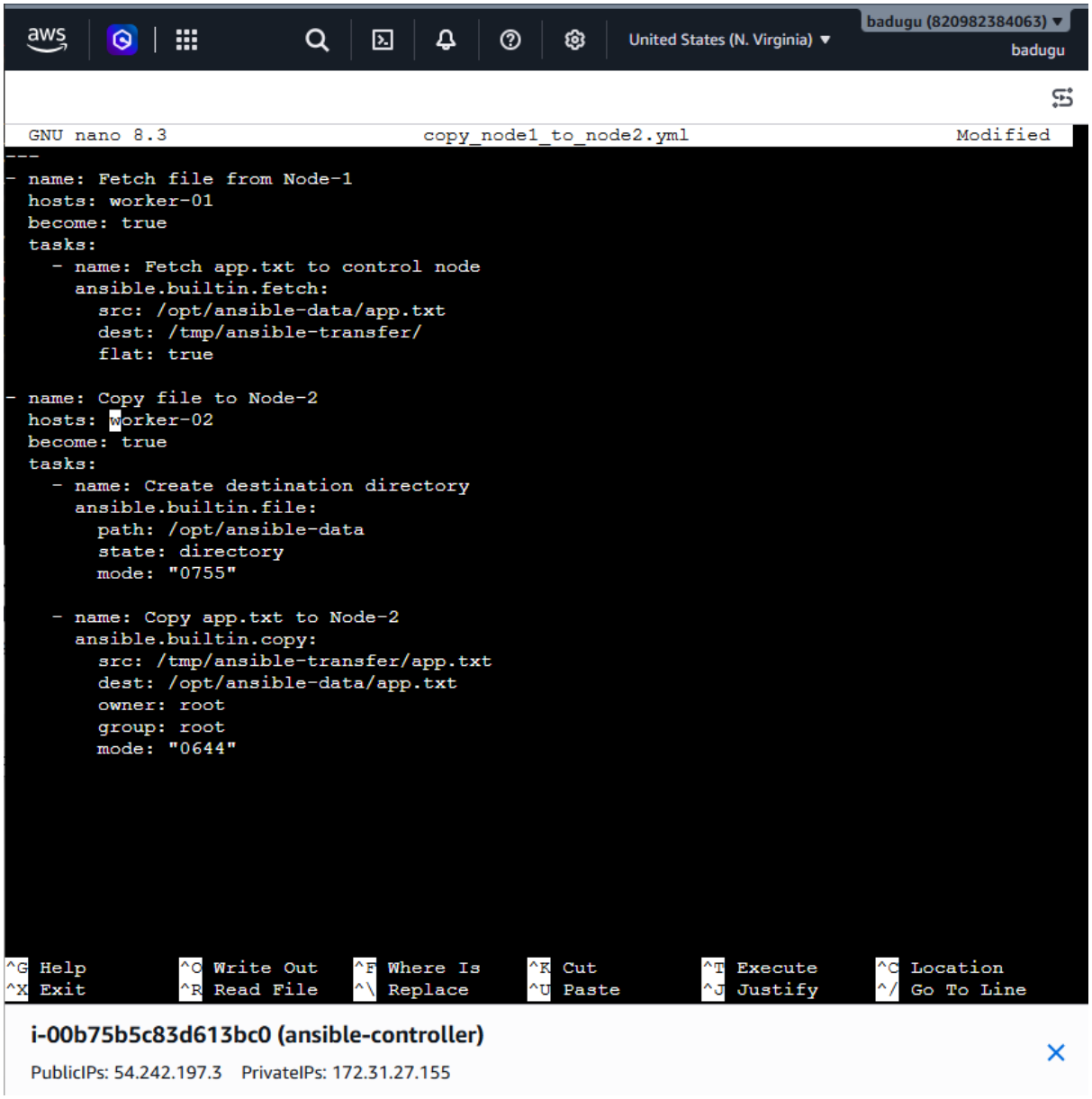


Run playbook: # ansible-playbook source_file.yml



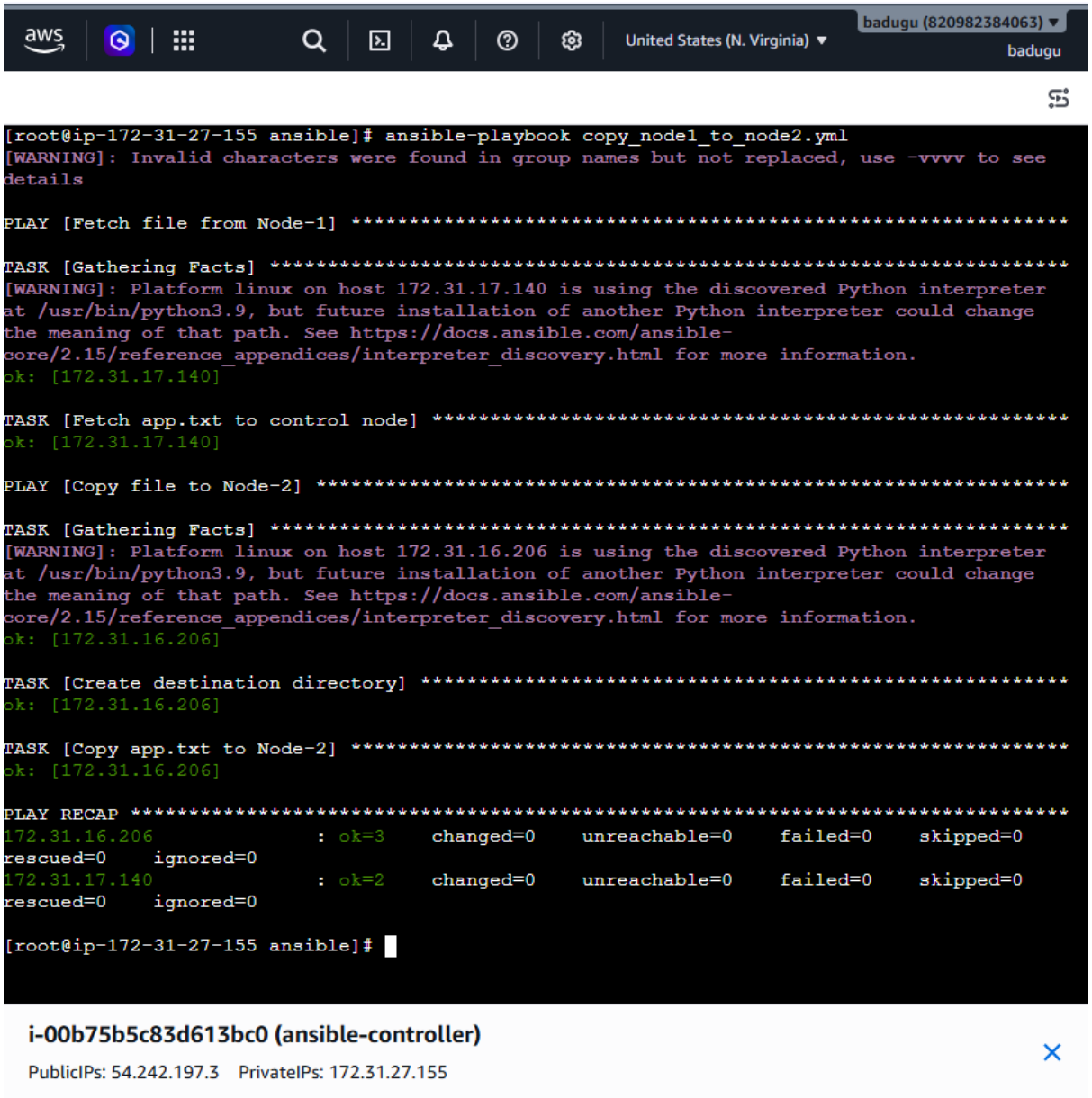
Step 4.2: Fetch the file from Node-1

Create copy_node1_to_node2.yml:



Step 4.3: Execute the playbook

ansible-playbook copy_node1_to_node2.yml



The transfer takes this path:

Node-1

/opt/ansible-data/app.txt

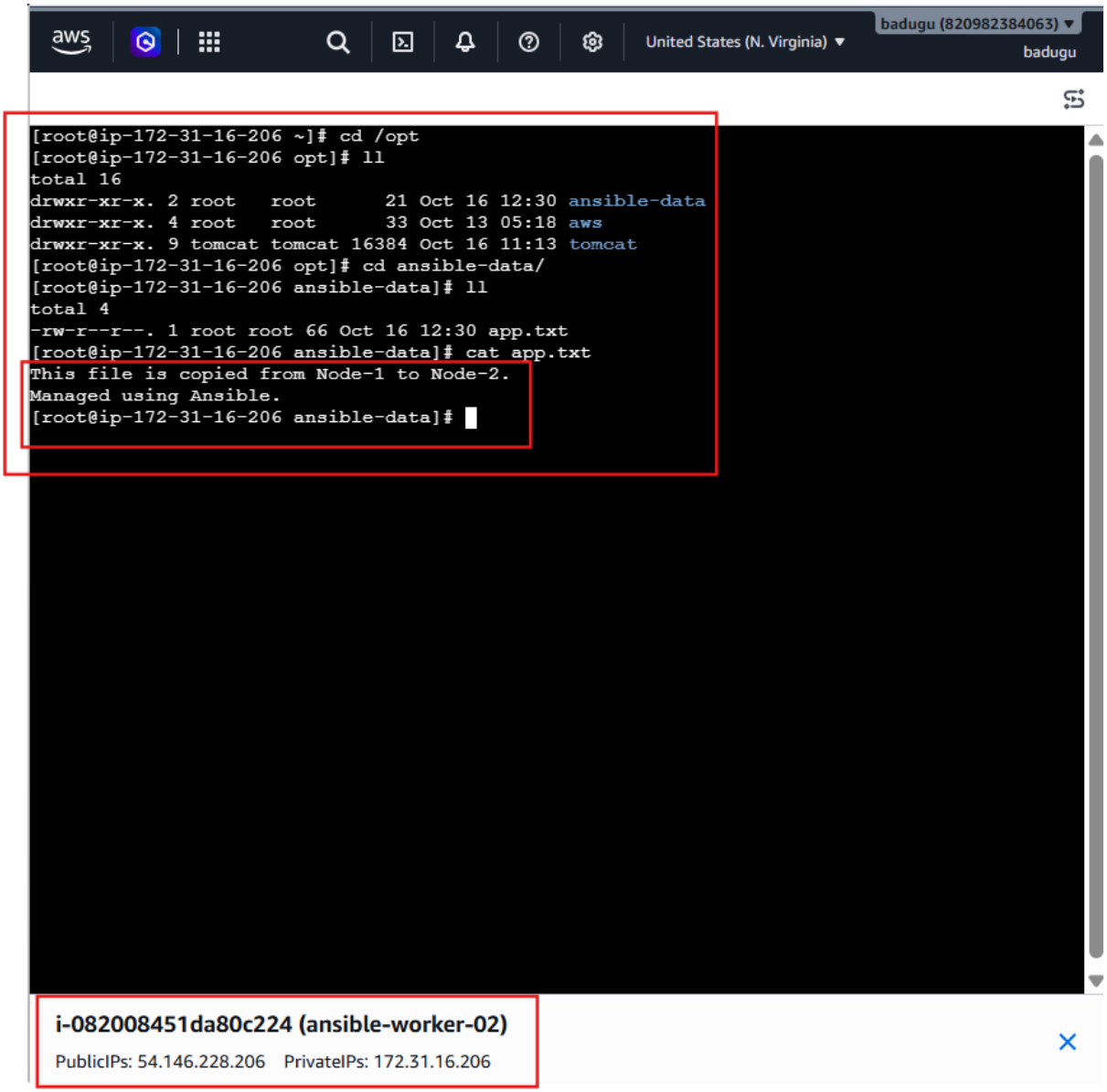
Ansible control node

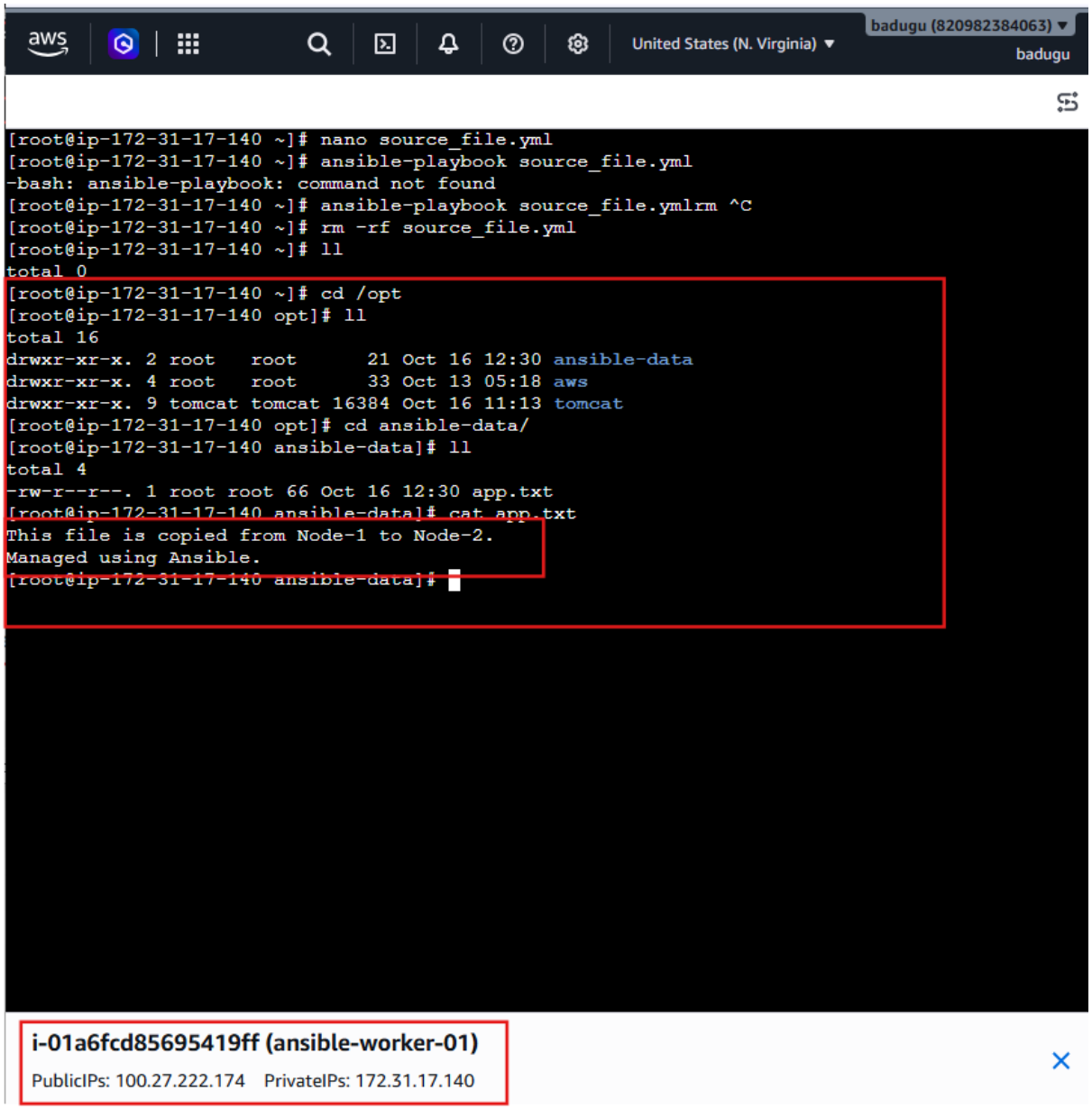
/tmp/ansible-transfer/app.txt

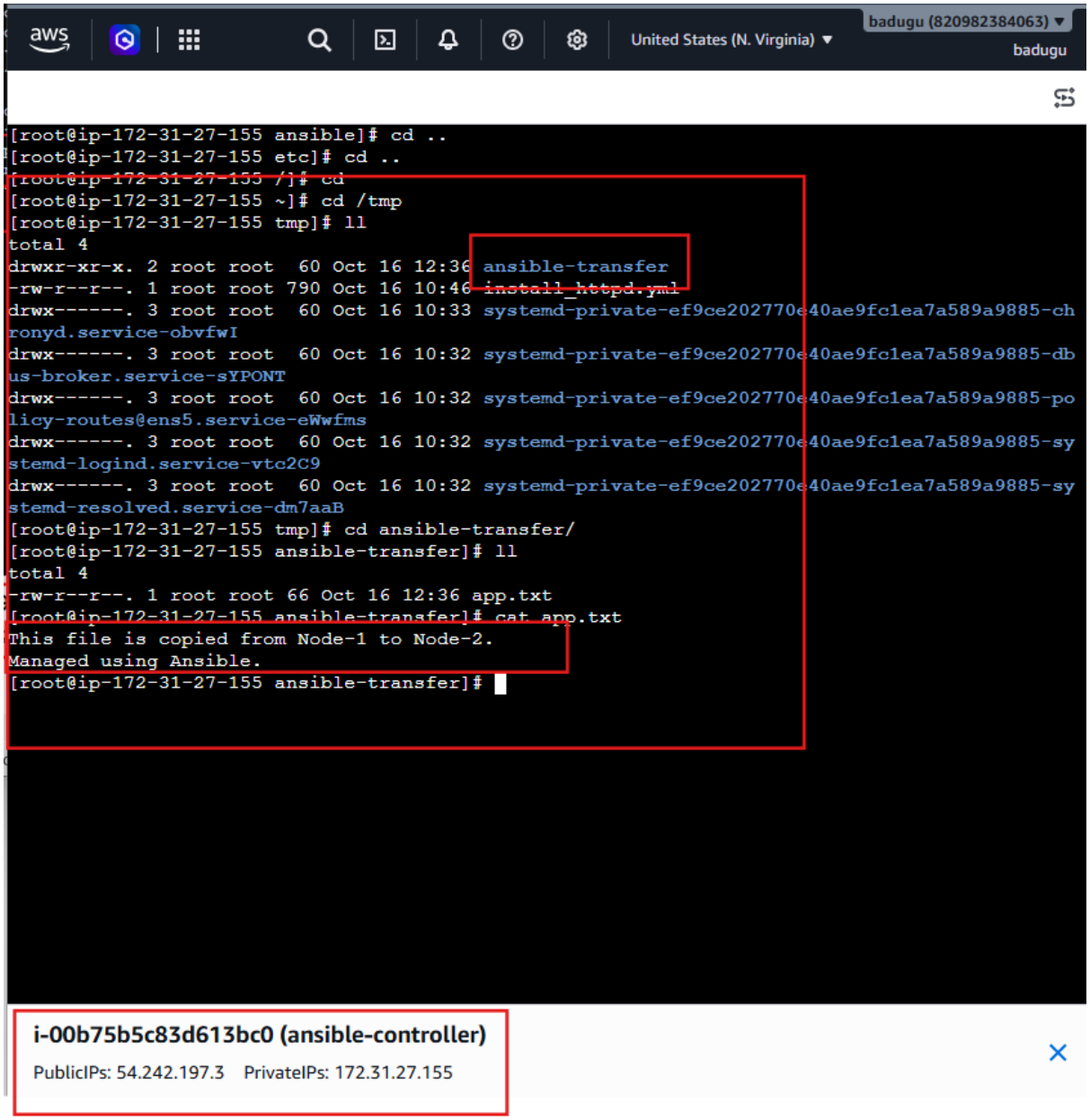
Node-2

/opt/ansible-data/app.txt

Step 4.4: Verify the copied file


aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-16-206 ~]# cd /opt
[root@ip-172-31-16-206 opt]# ll
total 16
drwxr-xr-x. 2 root root 21 Oct 16 12:30 ansible-data
drwxr-xr-x. 4 root root 33 Oct 13 05:18 aws
drwxr-xr-x. 9 tomcat tomcat 16384 Oct 16 11:13 tomcat
[root@ip-172-31-16-206 opt]# cd ansible-data/
[root@ip-172-31-16-206 ansible-data]# ll
total 4
-rw-r--r--. 1 root root 66 Oct 16 12:30 app.txt
[root@ip-172-31-16-206 ansible-data]# cat app.txt
This file is copied from Node-1 to Node-2.
Managed using Ansible.
[root@ip-172-31-16-206 ansible-data]#
i-082008451da80c224 (ansible-worker-02)
PublicIPs: 54.146.228.206 PrivateIPs: 172.31.16.206


aws
United States (N. Virginia)
badugu (820982384063)
badugu
[root@ip-172-31-17-140 ~]# nano source_file.yml
[root@ip-172-31-17-140 ~]# ansible-playbook source_file.yml
-bash: ansible-playbook: command not found
[root@ip-172-31-17-140 ~]# ansible-playbook source_file.ymlrm ^C
[root@ip-172-31-17-140 ~]# rm -rf source_file.yml
[root@ip-172-31-17-140 ~]# ll
total 0
[root@ip-172-31-17-140 ~]# cd /opt
[root@ip-172-31-17-140 opt]# ll
total 16
drwxr-xr-x. 2 root root 21 Oct 16 12:30 ansible-data
drwxr-xr-x. 4 root root 33 Oct 13 05:18 aws
drwxr-xr-x. 9 tomcat tomcat 16384 Oct 16 11:13 tomcat
[root@ip-172-31-17-140 opt]# cd ansible-data/
[root@ip-172-31-17-140 ansible-data]# ll
total 4
-rw-r--r--. 1 root root 66 Oct 16 12:30 app.txt
[root@ip-172-31-17-140 ansible-data]# cat app.txt
This file is copied from Node-1 to Node-2.
Managed using Ansible.
[root@ip-172-31-17-140 ansible-data]#
i-01a6fcd85695419ff (ansible-worker-01)
PublicIPs: 100.27.222.174 PrivateIPs: 172.31.17.140



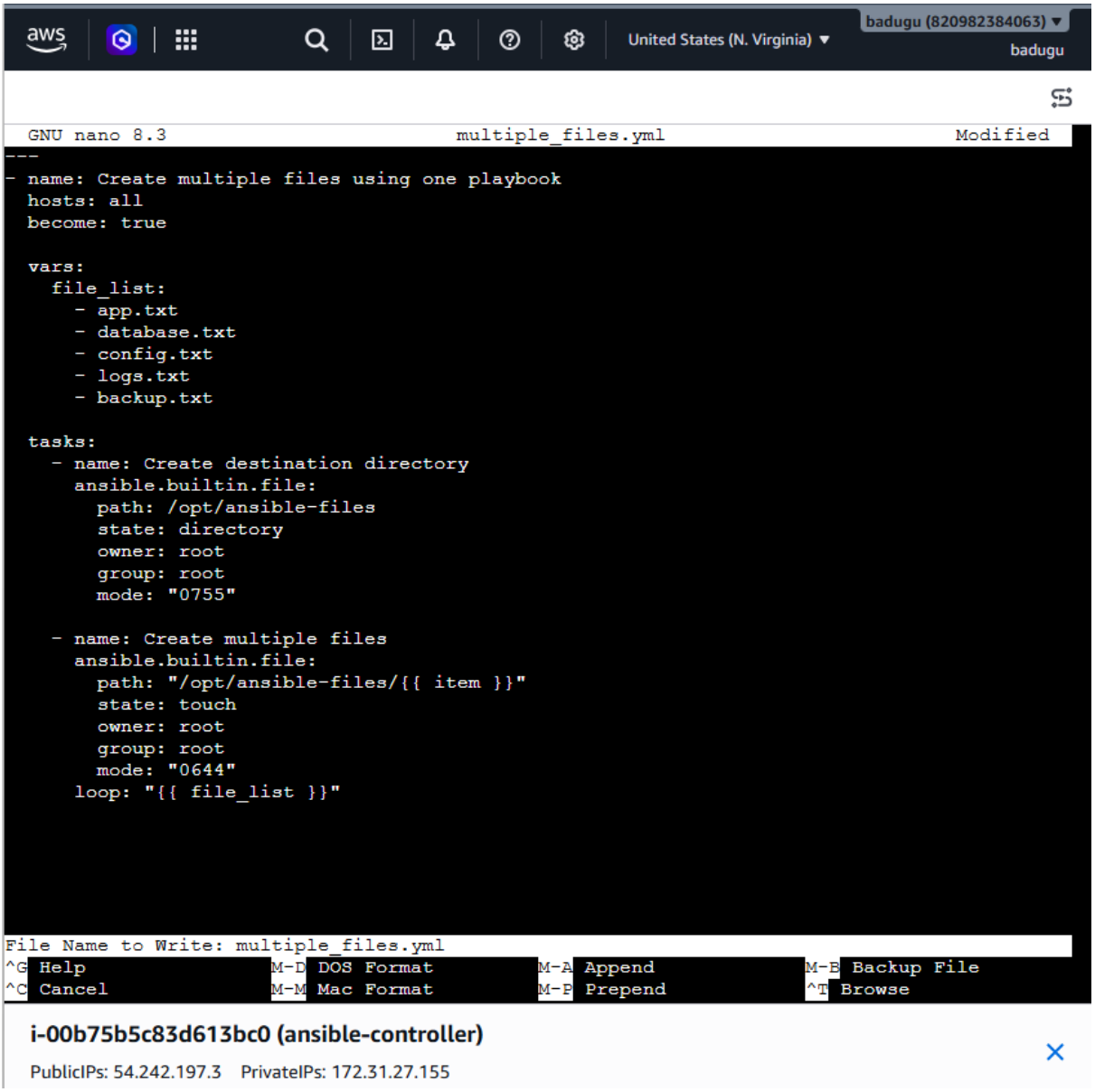
Interview tip: The copy module transfers files from the control node to a managed node by default. The fetch module transfers files from a managed node to the control node. Combining them lets you transfer a file between two managed nodes.

5.Write a ansible playbook to create different files with different names using single playbook.

Create different files with different names using a single Ansible playbook

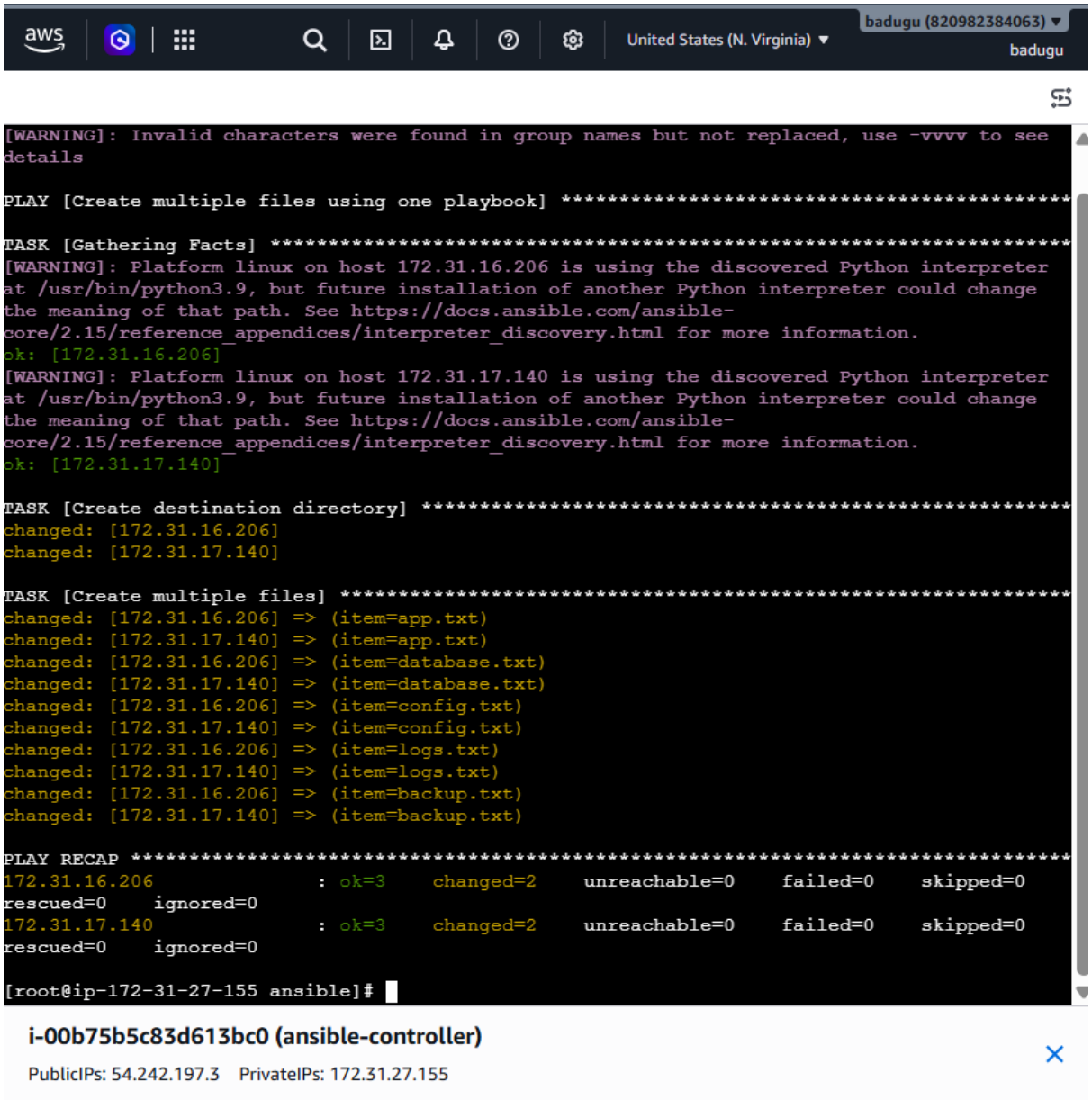
You can use a loop to create multiple files without writing a separate task for each file.

Step 5.1: Create create_multiple_files.yml

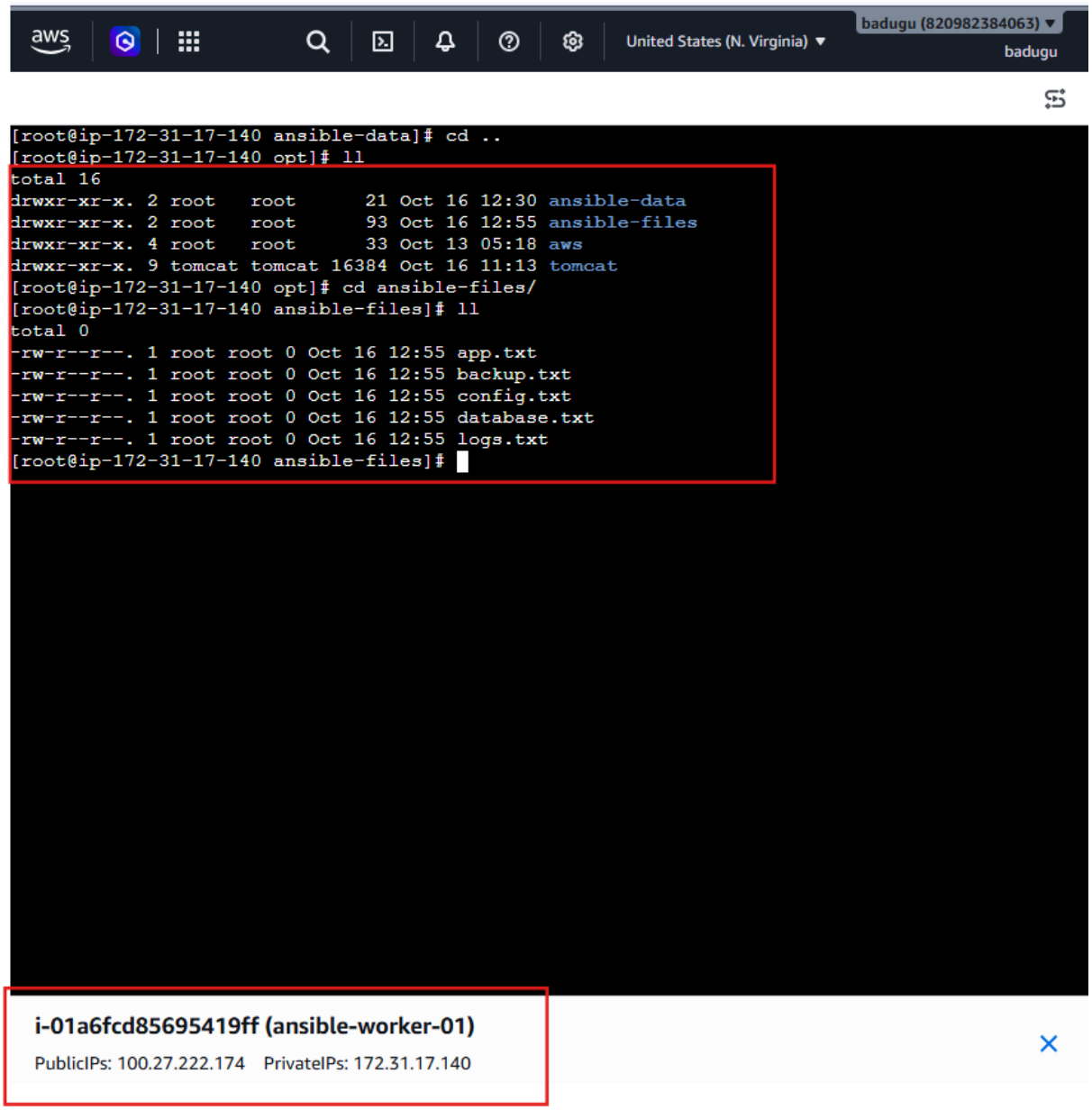


Step 5.2: Run the playbook

ansible-playbook multiple_files.yml

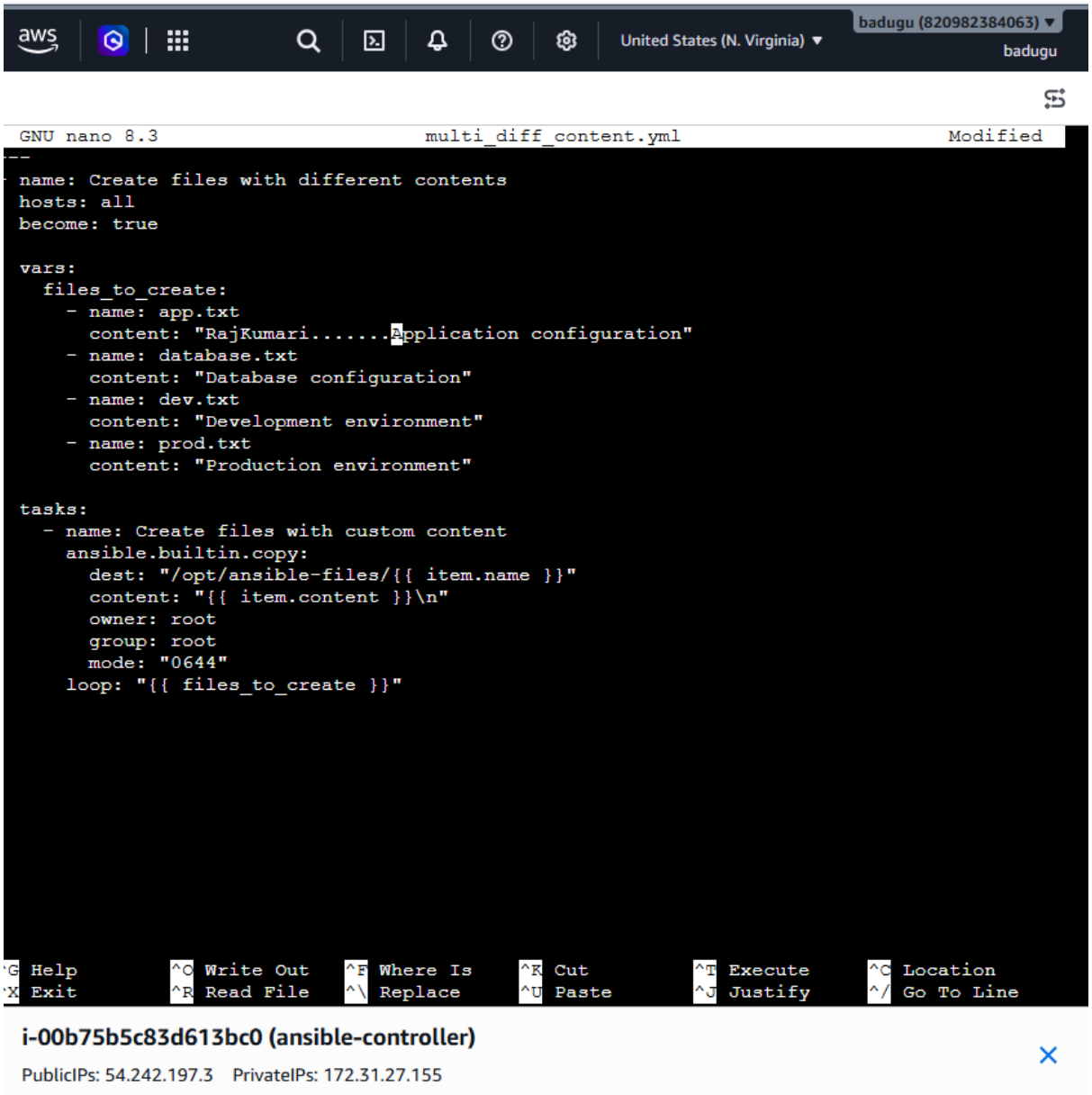


Step 5.3: Verify all files



Step 5.4: Create files with different names and different contents

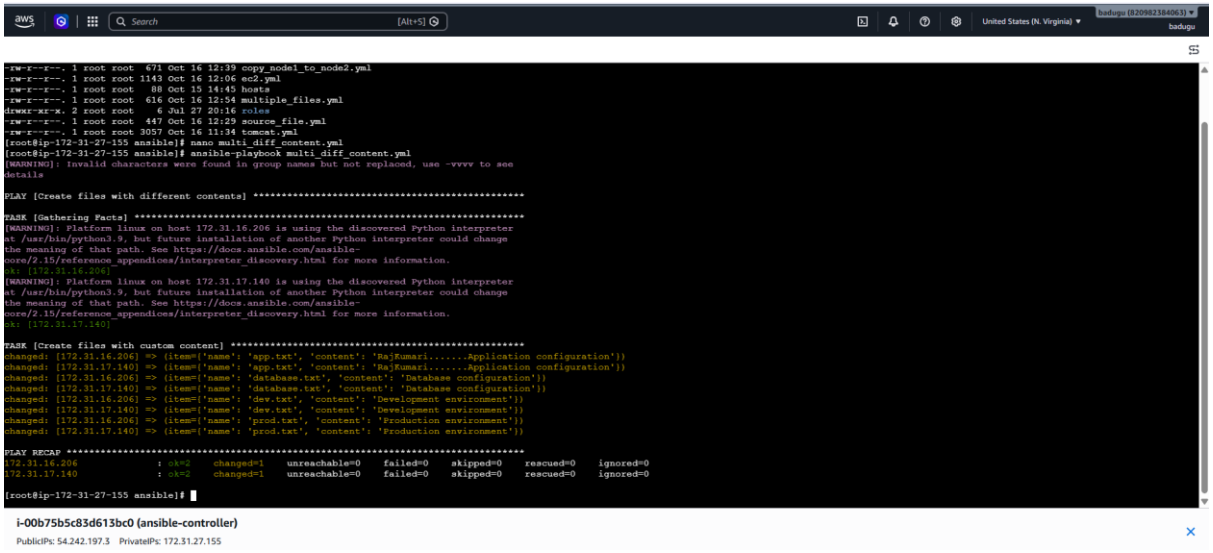
If each file needs different content, use a list of dictionaries:



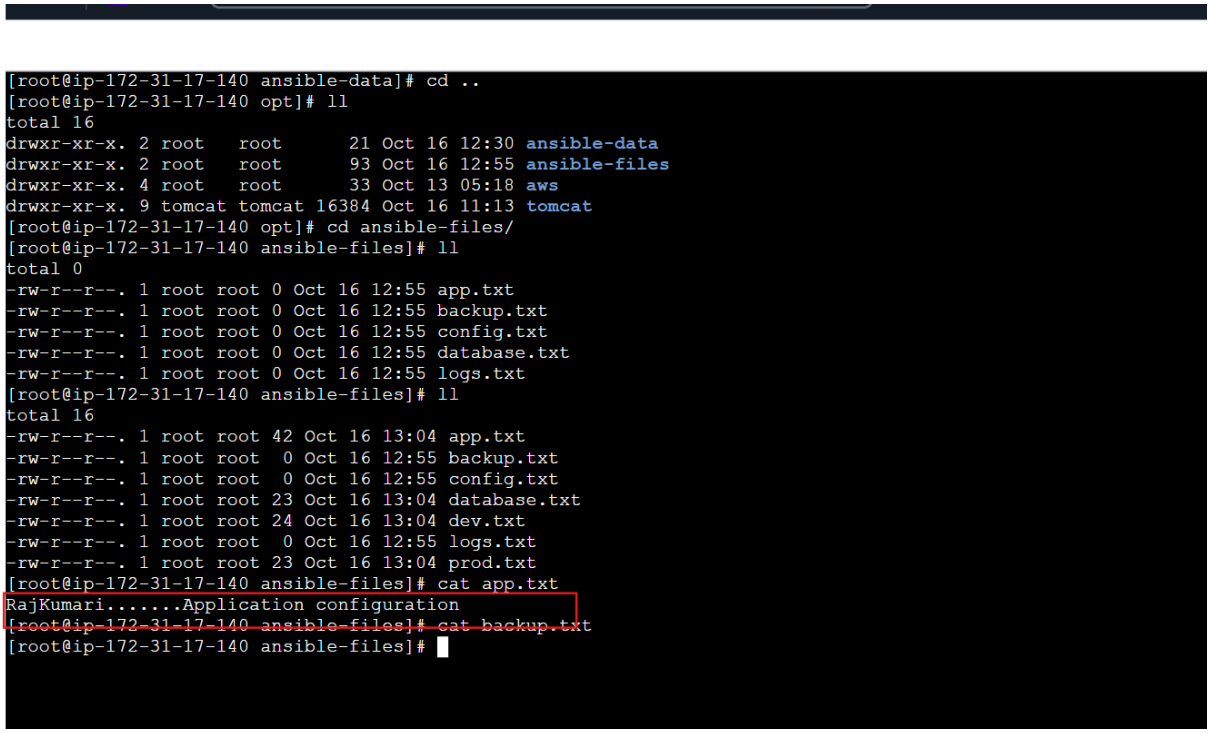
Run the same command:

ansible-playbook multi_diff_content.yml

If you use this second version, replace the first playbook's contents with the new version before running it.



Verify:



Final revision: All five playbooks

No.	Task	Main Ansible modules/concepts
1	Install HTTPD	dnf, copy, service, notify, handlers
2	Install Tomcat	dnf, user, get_url, unarchive, systemd_service
3	Provision EC2	amazon.aws.ec2_instance, AWS credentials

4	Copy file Node-1 → Node-2	fetch, copy, multi-play workflow
5	Create multiple files	file, copy, loop, variables